

Nectar Senior Java Developer Challenge

Position

Role: Senior Java Developer

Experience: 4–6 Years

Location: Coimbatore, Tamil Nadu, India

Open Positions: 1

About Nectar

Nectar is an AI-first product company building intelligent platforms that transform how enterprises leverage AI, automation, and connected technologies.

Our platform combines **Generative AI, Agentic AI, IoT, workflow automation, event-driven architecture, and cloud-native technologies** to deliver intelligent, scalable, and secure enterprise experiences.

As a Senior Java Developer at Nectar, you will work on backend systems supporting enterprise workflow management, automation, asset management, and operational processes.

This assessment is designed to evaluate your ability to design and implement a **production-ready, scalable, secure, multi-tenant, event-driven workflow management platform**.

Challenge Overview

You are working on Nectar's **Enterprise Workflow Management Platform**.

Multiple organizations use the platform to define and execute their own business workflows.

Different tenants may have completely different workflow definitions.

The workflow must therefore be **configurable and data-driven**.

The platform should also support an **event-driven architecture** where workflow and task activities can generate events and asynchronous processing can be introduced where appropriate.

Task 1: Core Java, OOP & Domain Design

Design the workflow platform using strong **object-oriented programming principles**.

The implementation should demonstrate practical understanding of:

- Encapsulation
- Abstraction
- Inheritance
- Polymorphism
- Interfaces
- Composition
- SOLID principles
- Design patterns
- Java Collections
- Generics
- Exception handling
- Immutability where appropriate

Domain Modeling

Design an appropriate domain model for:

- Tenant
- User
- Role
- Project
- Workflow
- Workflow State
- Workflow Transition
- Workflow Action
- Task
- Task Assignment
- Task Comment
- Task History
- Approval
- Audit Record

The candidate should determine the appropriate relationships between these objects.

Design Patterns

Identify and use appropriate design patterns where required.

Examples may include:

- Strategy
- Factory
- State
- Observer
- Template Method
- Builder

The candidate must explain the reason for choosing any design pattern.

Guide: Focus on maintainability and appropriate abstraction. Do not use design patterns simply to demonstrate patterns.

Task 2: Multi-Tenant Workflow & PostgreSQL Architecture

Design a **multi-tenant SaaS workflow platform**.

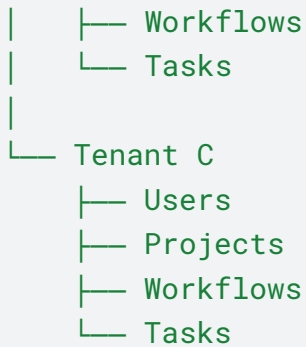
The system must support multiple independent organizations.

Example:

None

Nectar Platform





Tenant Isolation

Design a mechanism to identify the tenant for every request.

For example, tenant information could originate from:

- Request metadata
- Authentication context
- JWT claims
- Another appropriate mechanism

Security Requirement

The system must ensure that Tenant A cannot:

- Read Tenant B data
- Update Tenant B data
- Delete Tenant B data
- Access Tenant B workflow definitions
- Access Tenant B users
- Access Tenant B task history

The candidate must describe and demonstrate how this is prevented.

Hibernate/JPA

Demonstrate appropriate use of:

- Entity relationships
- Lazy/eager loading

- Cascading
- Entity lifecycle
- Optimistic locking
- JPQL/native queries where appropriate

The candidate should consider potential issues such as:

- N+1 queries
- Large object graphs
- Lazy loading problems
- Unnecessary eager loading
- Improper cascading

Multi-Tenant Architecture

Compare possible approaches such as:

- Shared database / shared schema
- Schema per tenant
- Database per tenant

Select an approach and explain the trade-offs.

Guide: The objective is not to select a predetermined architecture. We want to understand the candidate's reasoning around **security, scalability, operational complexity, cost, and maintainability**.

Task 3: Configurable Workflow & Execution Engine

Build a configurable workflow management system.

Workflows must not be hard-coded.

A workflow should support concepts such as:

None

Workflow

↓

States

↓
Transitions
↓
Conditions
↓
Actions

Workflow Definition

The system should allow authorized users to:

- Create workflows
- Update workflows
- Activate/deactivate workflows
- Define states
- Define transitions
- Define permitted roles
- Define workflow actions
- Configure approval steps

Workflow Execution

Implement the ability to execute a workflow transition.

For example:

None
Current State
↓
Requested Transition
↓
Validate
↓
Execute
↓
Next State

The system should consider:

- Whether the transition is valid
- Whether the user is authorized
- Whether required conditions are satisfied
- Whether required actions are completed
- Whether the state change is valid

Approval Workflow

The system should support workflows such as:



Support appropriate functionality for:

- Approve
- Reject
- Rework
- Approval comments
- Approval history
- Multiple approval levels
- Role-based approvals

Task Management

Tasks should support information such as:

- Task ID
- Tenant
- Title
- Description

- Priority
- Created by
- Assigned user
- Workflow
- Current state
- Due date
- Created timestamp
- Updated timestamp

Task 4: Event-Driven Architecture — Kafka & RabbitMQ

This is a **major part of the assessment**.

The workflow platform should support asynchronous and event-driven processing.

The candidate must demonstrate understanding of both:

- **Apache Kafka**
- **RabbitMQ**

The candidate should determine where each technology is appropriate.

Kafka Requirement

Design and implement an event flow for workflow-related events.

Potential events include:

None

TaskCreated

TaskAssigned

TaskTransitioned

TaskApproved

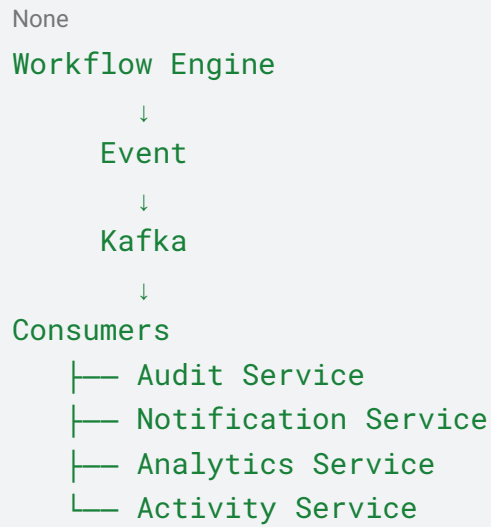
TaskRejected

TaskCompleted

WorkflowActivated

WorkflowUpdated

For example:



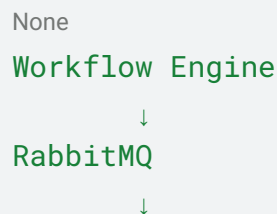
The candidate should consider:

- Topic design
- Partitioning
- Consumer groups
- Message ordering
- Event keys
- Offset management
- Consumer failure
- Retry handling
- Duplicate events
- Idempotency

RabbitMQ

Design and implement an asynchronous task-processing flow using RabbitMQ.

Possible use cases include:



Worker



Process Action

Potential use cases may include:

- Notifications
- Background workflow actions
- Task processing
- Retryable operations
- Scheduled/background jobs

The candidate should demonstrate understanding of:

- Exchanges
- Queues
- Routing keys
- Bindings
- Acknowledgements
- Retry mechanisms
- Dead-letter queues
- Message failure
- Consumer behaviour

Task 5: Security, Transactions, Concurrency & Reliability

Design the system for production-level reliability.

Authentication & Authorization

Implement or design appropriate authentication and authorization.

At minimum, support roles such as:

None

ADMIN

MANAGER

EMPLOYEE

Example responsibilities:

None

ADMIN

- └─ Manage Users
- └─ Manage Roles
- └─ Manage Workflows

MANAGER

- └─ Assign Tasks
- └─ Approve Tasks
- └─ View Team Tasks

EMPLOYEE

- └─ View Assigned Tasks
- └─ Update Tasks
- └─ Submit Tasks

Authorization must be enforced by the backend.

Transaction Management

Consider a workflow transition that performs several operations:

None

Task Update

↓

State Update

↓

History Creation

↓

Approval Update

↓

Audit Creation



Event Publishing

The candidate should determine appropriate transaction boundaries and explain how consistency is maintained.

Concurrency

The candidate must prevent inconsistent workflow state.

Consider appropriate approaches such as:

- Optimistic locking
- Pessimistic locking
- Database constraints
- Transaction isolation

The candidate should explain the chosen approach.

Reliability & Message Processing

The event-driven architecture must consider:

- Duplicate messages
- Message loss
- Consumer failures
- Producer failures
- Retry
- Dead-letter handling
- Idempotency
- Ordering
- Partial failures

The candidate should explain how the system behaves when:

None

Database Update



Event Publishing X

or:

None

Message Published ✓

Consumer Processing X

The objective is to evaluate the candidate's understanding of **distributed-system reliability**.

Task 6: Production Deployment, Apache Tomcat, Docker, Testing & Scalability

Prepare the application for a production-like environment.

Apache Tomcat

The application must run on **Apache Tomcat**.

The candidate should demonstrate understanding of:

- Application deployment
- Embedded Tomcat
- External Tomcat deployment
- WAR packaging where applicable
- Server configuration
- Application configuration

The candidate should document the selected deployment approach and explain why it was chosen.

Docker

Containerize the application.

Provide:

None

Dockerfile

docker-compose.yml

The candidate should configure the services appropriately.

Testing

Implement meaningful automated tests.

Unit Tests

Test important business logic including:

- Workflow transitions
- Permission validation
- Approval logic
- Task operations
- Workflow rules

Integration Tests

Test:

- REST APIs
- PostgreSQL
- Hibernate
- Multi-tenancy
- Transactions
- Security

Messaging Tests

Test appropriate Kafka and RabbitMQ scenarios.

Consider:

- Message publishing
- Message consumption
- Duplicate messages
- Consumer failure

- Retry
- Dead-letter handling

Performance & Scalability

Assume the platform eventually needs to support:

None

10,000+ Tenants

1,000,000+ Tasks

Millions of Workflow History Records

Thousands of Concurrent Users

Large Event Volumes

Explain how the architecture could scale.

Consider:

- PostgreSQL indexing
- Query optimization
- Connection pooling
- Pagination
- Caching
- Kafka partitioning
- Kafka consumer scaling
- RabbitMQ consumer scaling
- Database scaling
- Horizontal application scaling
- Stateless services
- Asynchronous processing

The candidate does not need to implement every optimization.

Bonus Challenges

The following are optional and can be used to distinguish strong candidates.

Bonus 1: Outbox Pattern

Design or implement a solution to reliably handle:

None

Database Transaction

+

Event Publishing

Explain how you would prevent a situation where the database transaction succeeds but the event is lost.

Bonus 2: Event Replay

Design a mechanism to replay historical Kafka events.

Explain:

- Event versioning
- Idempotency
- Consumer state
- Replay safety

Bonus 3: Redis

Use Redis for appropriate use cases such as:

- Workflow configuration caching
- Tenant configuration
- Frequently accessed data

Explain cache invalidation.

Expected Deliverables

Please submit:

1. Source Code
2. Database Design
3. Architecture Documentation
4. API Documentation
5. Test Cases

Evaluation Criteria

Criteria	Weightage
Core Java & OOP	12%
SOLID & Design Patterns	8%
Spring Boot Architecture	8%
Multi-Tenant Architecture	12%
PostgreSQL & Hibernate/JPA	10%
Workflow & Approval Engine	15%
Apache Kafka	10%
RabbitMQ	8%
Security, Transactions & Concurrency	7%
Testing & Code Quality	4%
Apache Tomcat & Docker	3%
Scalability & Architecture	3%
Total	100%

What We Are Looking For

This assignment is not intended to evaluate only basic Java and Spring Boot CRUD development.

We want to understand whether the candidate can work as a **Senior Backend Engineer** on a production enterprise platform.

Submission Timeline

Please submit the completed assignment within **5 calendar days** from the date of receiving the challenge.

Important Notes

- Clearly document assumptions wherever applicable.
- Do not hard-code tenant information.
- Do not hard-code workflow business logic.
- Apache Tomcat deployment must be addressed.
- Docker support is mandatory.
- Automated tests are expected.
- Security and tenant isolation are mandatory.
- Candidates are encouraged to explain trade-offs rather than over-engineer the solution.
- Bonus features are optional.
- Candidates may use AI coding assistants, but they must be able to explain all submitted code and architectural decisions.
- The candidate is not expected to implement every possible production feature. The quality of architecture, reasoning, implementation, and trade-offs is more important than the number of features.

We are interested not only in the final implementation, but also in the candidate's **engineering approach, reasoning, architecture, design decisions, code quality, and understanding of production distributed systems**.

We look forward to reviewing your submission.

Team Nectar